

做一个自动化的超低频资金费率和基差套利工具，因为只有个人开发，个人使用，不在乎性能（延迟有个几秒都能接受），代码简单易懂，不要以重型项目的标准去做。采用单进程+Goroutine，而不是多个微服务。不要使用 jwt 鉴权，前端页面无需跨域，简单的本机 localhost 即可。前端页面要求简洁。

对于一个个人小型量化工具足够了。

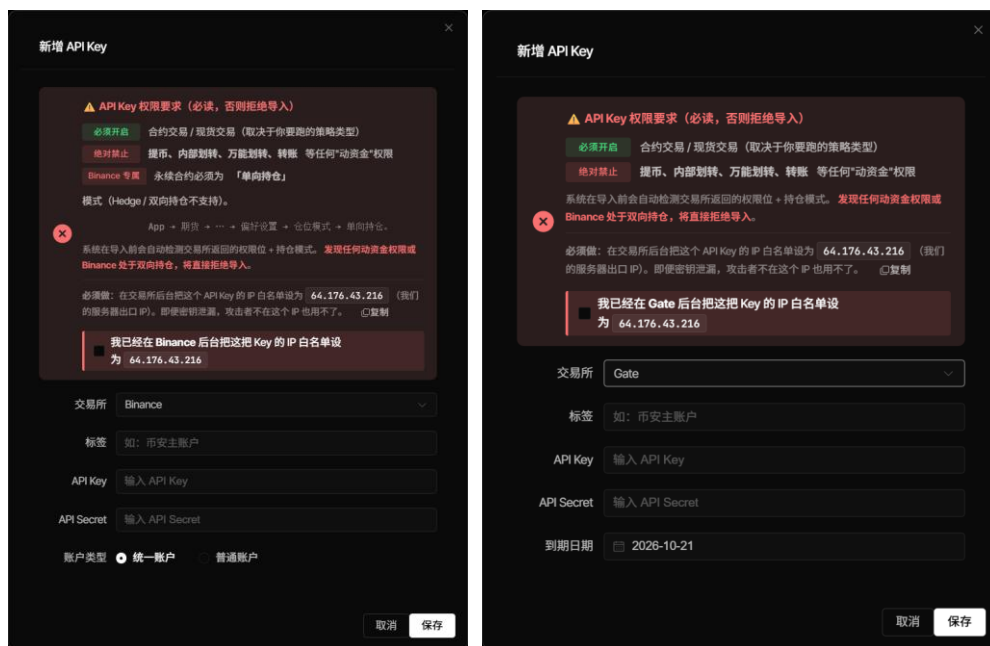
使用 golang，使用 cctx，连接 biance 和 gate。

1、API 配置模块

模块基础定义：测试交易所的 API 连通性与权限可行性。

UI 界面：参考的别人的项目，非我的要求，仅供参考





调用者：请求响应式，前端调用该模块 API，传入各个交易所的 API key、密钥，保存到数据库。前端调用验证 API，返回验证结果

输入数据：

来源：前端主动填写

数据名称：交易所名称、APIKEY、Secret

输出数据：前端显示调用结果，保持成功、保持失败、测试连接成功、测试权限

2、行情模块

模块基础定义：展示待买入列表

UI 界面：参考的别人的项目，非我的要求，仅供参考

Symbol	Portfolio	Funding Rate	PNL 3 Day Cum.	Funding Rate	3 Day Revenue
 ESPTS	Buy  ESPTS/USDT Spot	0.2384%	\$23.84	1.8776%	\$187.76
	Short  ESPTS/USDT Perpetual				
 龙虾	Buy  龙虾/USDT Spot	0.0194%	\$1.94	0.8466%	\$84.66
	Short  龙虾/USDT Perpetual				
 FHE	Buy  FHE/USDT Spot	0.0250%	\$2.50	0.8152%	\$81.52
	Short  FHE/USDT Perpetual				
 SIREN	Buy  SIREN/USDT Spot	0.0856%	\$8.56	0.7068%	\$70.68
	Short  SIREN/USDT Perpetual				

币种	合约交易所	合约费率	持仓	合约价格	GATE持仓	HEX持仓	BITGET持仓	BYBIT持仓	基差率	24H合约量	持仓方向	操作按钮
AEROO	Binance	0.5818%	1274.2%	0.0231	--	--	--	--	--	3.9M	空Binance合约 / 多Gate现货	创建任务
KSTR	Binance	0.5348%	585.6%	25.44	--	--	25.44 KSTR 24.8M	--	--	3.4M	空Binance合约 / 多Gate现货(RKSTR)	创建任务
ESPTS	Binance	0.0324%	283.8%	0.03131	0.03402 2218K	0.0408 66.9K	--	--	-7.966%	470M	空Binance合约 / 多Gate现货	创建任务
SIREN	Binance	0.1243%	272.3%	0.03085	0.0617 309.3K	0.0363 95.9K	--	--	-15.014%	71M	空Binance合约 / 多MEXC现货	创建任务
TRUTH	Binance	0.0630%	138.0%	0.012522	--	0.012459 194.3K	--	--	+0.506%	2.3M	空Binance合约 / 多MEXC现货	创建任务
SPORTFUN	Binance	0.0457%	100.1%	0.02335	--	--	--	--	--	933.3K	空Binance合约 / 多Gate现货	创建任务

调用者：请求响应式

- 1、前端刷新页面或者点击刷新按钮调用该模块 API 获取最新行情
- 2、自动交易模块（后续会讲到）调用该模块 API 获取最新行情

输入数据：

数据名称：各个币种合约的最近 N（5）个 FundingRate、基差 Basic、24H（现货和合约）流通量

来源：Binance、Gate（只是举例而已，事实上一个交易所多现货，一个交易所空合约，甚至同一个交易所多现货空合约都可以，和交易所的连接要抽象化，方便更换交易所）

数据名称：N、流通量、基差等各种约束的值设置，来自于设置模块传入数据库的值，在数据库中读取。

来源：用户在前端设置页面手动设置

核心处理逻辑：

因为我们要在 **Binance** 买合约、**Gate** 买现货，所以将 **binance** 的合约和 **gate** 的对应的现货（如果有的话）根据以下约束进行过滤后，返回列表。这些列表中的每一个币种对冲交易对，都是优质的下单选项，如果没有通过过滤的币种，这张表就空着。

1、合约和现货之间的基差约束（大于 0.1%，合约比现货贵）

2、合约的资金费率满足趋势上升约束：前 $N < \text{后 } N$ （也就是说在上升的里面，选择资金费率表现好的）

2、资金费率 **AVG** 约束，最近 N 次 **funding** 的 **AVG** 大于 0.05%

3、现货和合约满足**流通量**约束（大于 50K）

异常与容错处理：

接收者：前端调用者、自动交易模块

输出数据：币种、来源交易所

模块设计考量与权衡：

1、是拿到全部的原生数据再筛选，还是调用 **api** 就可以筛选？这部分可以优化。

2、交易所成千上百个，为什么只选择 2 个交易所（一个现货、一个合约）？因为每个交易所的情况都大同小异，相似度超过 90% 以上，区别也主要是深度的区别，没必要七八个交易所，换来换去，来回转账。每个交易所的手续费返佣、VIP 等级都需要积累的。频繁更换交易所，费时费力最后赚的额外收益被手续费吞掉一大半。而

且自动程序会变得十分困难，2 个交易所一个负责合约、一个负责现货，实现起来，很方便。

3、交易模块

模块基础定义：交易模块，建仓平仓都用这个

UI 界面：参考的别人的项目，非我的要求，仅供参考

基础配置

* 任务名称

如: BTC 费率套利 Binance-OKX

展示交易对

如 BTC/USDT (仅展示用)

数量模式

按币数

按USDT

关联策略

可选

完成后同步

☐

完成后自动创建新策略

腿A配置

优先腿

交易所

选择交易所

API Key

选择Key

方向

多

空

角色

优先

对冲

市场类型

合约

现货

交易对

输入搜索交易对

总数量

0.00

币

单次数量

0=一次全部

每次下单量 (空=一次全下)

下单方式

Taker

Maker

杠杆

-

10

+

保证金模式

全仓

逐仓

盘口档位

-

3

+

最大追价次数

-

50

+

超限转Taker

☐

掉出前N档自动追价到第1档

超过后转Taker

腿B配置

对冲腿

交易所

选择交易所

API Key

选择Key

方向

多

空

角色

优先

对冲

市场类型

合约

现货

交易对

输入搜索交易对

总数量

0.00

币

单次数量

0=一次全部

每次下单量 (空=一次全下)

下单方式

Taker

Maker

杠杆

-

10

+

保证金模式

全仓

逐仓

盘口档位

-

3

+

最大追价次数

-

50

+

超限转Taker

☒

掉出前N档自动追价到第1档

超过后转Taker

执行控制

优先/对冲

在每条腿的配置中选择“优先腿”或“对冲腿”。优先腿先执行，对冲腿在优先腿成交后按比例跟进。

对冲触发

实时对冲

累计百分比

最大净敞口

0

枚 (0=不限，超出后自动停止)

最大重试

-

3

+

净敞口

0.000000

总手续费

0.0000

腿数

2

优先腿

腿B

腿A — BINANCE		腿B — GATE		净敞口仪表盘
API Key	—	API Key	—	 0.00 USDT 700.0
市场类型	合约	市场类型	现货	
杠杆	10x cross	成交/总量	339,660 / 339,660	
单次数量	3,000	均价	0.0020987026	
手续费	0.00	均价	0.00209846245	
交易对	AKA/USDT/USDT	交易对	AKA/USDT	
下单方式	Maker	下单方式	Maker	
成交/总量	339,660 / 339,660	单次数量	3,000	
均价	0.0020987026	手续费	0.00	

订单ID	成交ID	价格	数量	手续费	触发对冲
78475	137287991	0.0021101	3,000	0.001266060000 USDT	false
78473	137287713	0.0021076	3,000	0.001264560000 USDT	false
78471	5489207	0.0021049	3,000	0.006314700000 USDT	false
78470	137287233	0.002108	3,000	0.001264800000 USDT	false
78469	5489198	0.0021077	3,000	0.006323100000 USDT	false

23:51:38	info	order_placed	腿A Maker挂单 buy 3000.0@0.0021024 (前3档)
23:51:38	info	hedge_triggered	第97轮 对冲腿A 下单 3000.0
23:51:38	info	filled	腿B 本轮成交 3000.0/3000.000000000000
23:51:35	info	order_placed	腿B 追价#1 3000.000000000000@0.0020946
23:51:35	info	order_placed	腿B 订单@0.002094600000 已掉出前3档 (当前最优: 0.0020922), 准备追价
23:51:31	info	order_placed	腿B Maker挂单 sell 3000.000000000000@0.0020946 (前3档)
23:51:28	info	order_placed	第97轮 优先腿B 下单 3000.000000000000 (已成交287091.0/339660.000000000000)
23:51:28	info	filled	腿A 本轮成交 3000.0/3000.0
23:51:28	info	order_placed	腿A Maker挂单 buy 3000.0@0.0020978 (前3档)
23:51:28	info	hedge_triggered	第96轮 对冲腿A 下单 3000.0
23:51:28	info	filled	腿B 本轮成交 3000.0/3000.000000000000
23:51:24	info	order_placed	腿B 订单@0.002097700000 已掉出前3档 (当前最优: 0.0020966), 准备追价
23:51:24	info	order_placed	腿B Maker挂单 sell 3000.000000000000@0.0020977 (前3档)
23:51:21	info	order_placed	第96轮 优先腿B 下单 3000.000000000000 (已成交284091.0/339660.000000000000)
23:51:21	info	filled	腿A 本轮成交 3000.0/3000.0

23:52:03	info	order_placed	腿A 订单@0.002096300000 已掉出前3档 (当前最优: 0.0020968), 准备追价
23:52:03	info	order_placed	腿A 追价#1 3000.0@0.0020963
23:52:03	warning	order_placed	腿A Maker单被拒(会立即成交), 追价#1
23:52:03	info	hedge_triggered	第100轮 对冲腿A 下单 3000.0
23:52:03	info	filled	腿B 本轮成交 3000.0/3000.000000000000
23:52:00	info	order_placed	腿B 追价#2 3000.000000000000@0.0020987
23:52:00	info	order_placed	腿B 订单@0.002100100000 已掉出前3档 (当前最优: 0.0020987), 准备追价
23:51:57	info	order_placed	腿B 追价#1 3000.000000000000@0.0021001
23:51:57	info	order_placed	腿B 订单@0.002100600000 已掉出前3档 (当前最优: 0.0020983), 准备追价
23:51:57	info	order_placed	腿B Maker挂单 sell 3000.000000000000@0.0021006 (前3档)
23:51:57	info	order_placed	第100轮 优先腿B 下单 3000.000000000000 (已成交296091.0/339660.000000000000)
23:51:57	info	filled	腿A 本轮成交 3000.0/3000.0
23:51:54	info	order_placed	腿A 追价#2 3000.0@0.0021039
23:51:54	info	order_placed	腿A 订单@0.002103500000 已掉出前3档 (当前最优: 0.0021039), 准备追价
23:51:50	info	order_placed	腿A 追价#1 3000.0@0.0021035

调用者：用户在前端发起、自动交易（后续会讲到）调用

输入数据：

来源：自动交易来自于设置模块传入数据库中的值，手动交易来自于用户手动填入，交易下单页面的默认设置值也是设置模块传入数据库中的值

数据名称：下单信息

核心处理逻辑：填入总量，然后每笔原子交易的量。如果是自动交易参数在设置模块设置的，手动下单则是默认同自动交易，但是可以手动在前端修改

异常与容错处理：参考 UI 页面中别人的信息，应该有成熟方案吧？

接收者：前端日志反馈

输出数据：参考 UI 页面中别人的信息

模块设计考量与权衡：

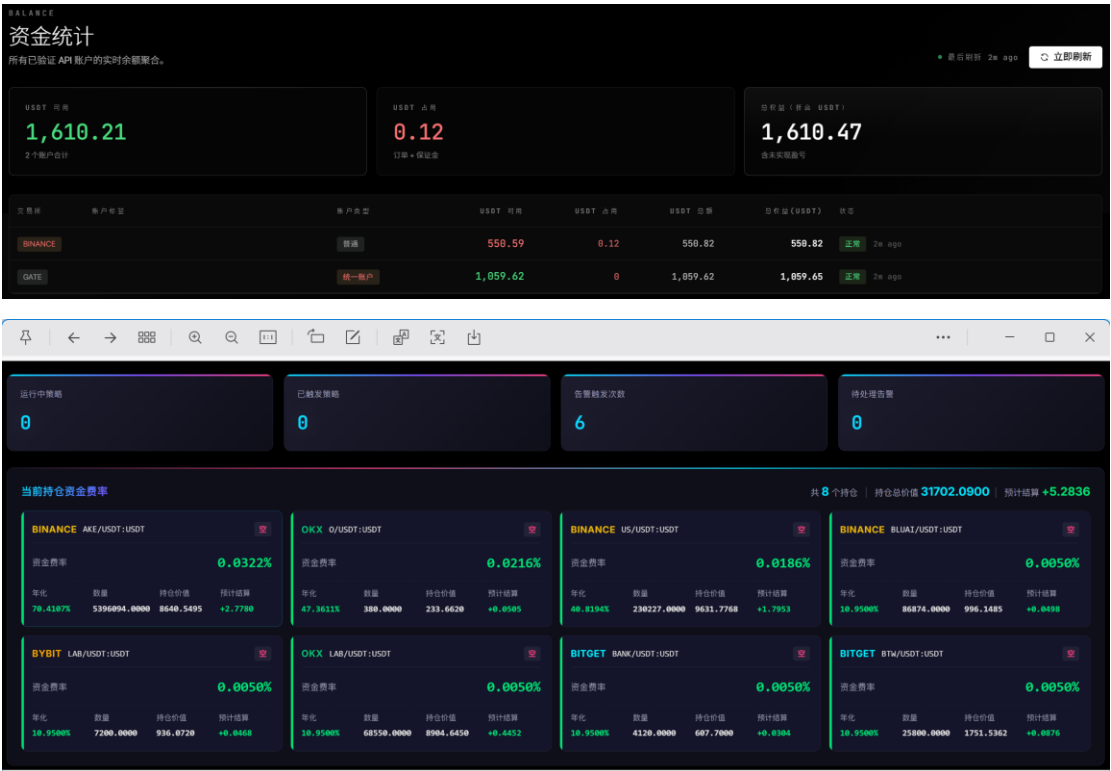
类似的下单引擎，应该有非常多了，不要自研。你的模型里面，或

者直接去 [github](#) 看看类似的知识。

同一个项目，多个模块，模块既能被前端调用，也能被其他后端模块调用。（为前端提供 RESTful API 接口，但后端模块之间（因为是单进程多 goroutine）直接通过导入包来调用。） 也就是说自动交易模块直接在后端调用交易模块就可以，后续自动交易模块所有调用到的其他模块，都是此方式。

4、账户信息模块

模块基础定义：binance 和 gate 各自账户的资金，持仓，现金等等等。以及聚合账户的总资金，历史收益，累计手续费等。 当前持有的交易对冲对有哪些。当前资金费率情况。累计收益，运行时间。 UI 界面（别人的项目，仅供参考）：



调用者：用户前端刷新、自动交易模块（后续会详细讲到）调用

输入数据：

来源：交易所

数据名称：参考 UI 中别人的项目

核心处理逻辑：

异常与容错处理：

接收者：前端页面、自动交易模块

输出数据：参考 UI 中别人的项目

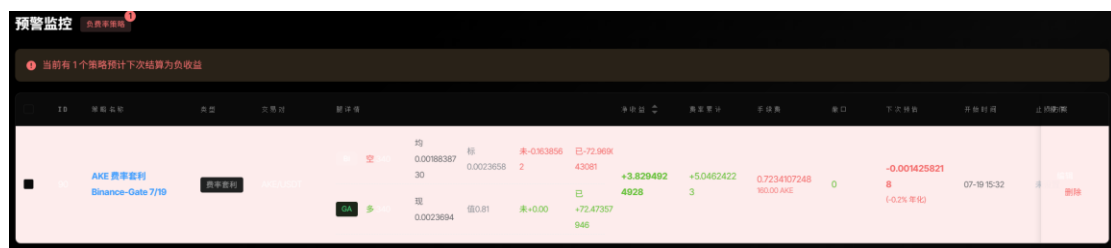
模块设计考量与权衡：

5、预警模块

模块基础定义：依赖账户信息模块，拿到当前持有的交易对，持续监听 ADL、资金费率反转（小于 0）、爆仓等，推送到邮箱。

还有一个就是资金平衡提示，比如 binance 和 gate，默认 4 倍杠杆，资金分配 1:4 如果双方误差超过 15%（可以在设置中调），提醒用户手动平衡资金（交易机器人，不能自动调仓，因为涉及到转账）

UI 界面（仅供参考）：



ADL 监控

运行中

轮询间隔 30s

告警阈值 ≥ 4

告警规则配置

告警阈值等级

等级 ≥ 4 (中风险告警)

当前: ≥ 4

重复提醒间隔

30 分钟

持续高风险时重复发邮件

轮询间隔 (秒)

-

30

+

建议 3~30 秒

保存配置

实时持仓 ADL 等级

交易所	交易对	方向	ADL 等级	持仓数量	更新时间
BINANCE	AKE/USDT:USDT	SHORT	1	340	2026/7/24 16:25:44
BINANCE	HANA/USDT:USDT	SHORT	1	12	2026/7/23 23:31:49

邮件推送

当前配置

发送测试邮件

编辑配置

SMTP 服务器

smtp.163.com

端口

465

发件邮箱

yl@****.com

收件人

yl@****.com

授权码

预警推送

已启用

推送记录

刷新

时间	预警数	收件人	主题	状态	操作
2026-07-24 15:56:10	1	yl@35364@gmail.com	[周末预警] 1个推地即将负费率结算	失败	删除
2026-07-24 15:53:16	1	yl@35364@gmail.com	[周末预警] 1个推地即将负费率结算	失败	删除
2026-07-24 15:48:32	1	yl@35364@gmail.com	[周末预警] 1个推地即将负费率结算	失败	删除
2026-07-24 15:43:17	1	yl@35364@gmail.com	[周末预警] 1个推地即将负费率结算	失败	删除
2026-07-24 15:38:56	1	yl@35364@gmail.com	[周末预警] 1个推地即将负费率结算	失败	删除

告警发送记录

告警类型	标题	内容	交易所	交易对	状态	确认状态	创建时间	操作
推送	[全币种] gate GT spot	查看			成功	已完成	2026-07-20 12:59:38	删除
推送	[全币种] gate GT spot	查看			成功	已完成	2026-07-20 12:59:38	删除
推送	[全币种] gate GT spot	查看	gate	GT	失败	未完成	2026-07-20 12:59:38	删除
推送	[ADL风控] bitget BANK/USDT:USDT SHORT ADL等级=5	查看			成功	已完成	2026-07-20 12:59:17	删除
推送	[ADL风控] bitget BANK/USDT:USDT SHORT ADL等级=5	查看			成功	已完成	2026-07-20 12:59:17	删除
推送	[ADL风控] bitget BANK/USDT:USDT SHORT ADL等级=5	查看	bitget	BANK/USDT:USD T	失败	未完成	2026-07-20 12:59:17	删除

共 6 条

20条/页

1

调用者：主动持续监听交易所信息，或者自动交易模块调用该模块

输入数据：

来源：交易所

数据名称：如 UI 所示

核心处理逻辑：

异常与容错处理：

接收者：自动交易、邮箱

输出数据：

模块设计考量与权衡：

6、自动交易模块

模块基础定义：轮询交易所，调用预警模块，账户信息，交易模块，行情模块等信息。

UI 界面：无

调用者：无

输入数据：

数据名称：

输入数据：设置模块的参数，

来源：用户在前端设置页面手动设置

数据名称：组容量（总量），原子单位

解释：比如以 50U（近似值，肯定没法恰好 50U）为一组买入卖出。这 50U 又分 5U 为原子单位（当前 50u 还是 5u 都是可以设置的）。调用交易引擎，输入总数量 50U，单次数量 5U。

举例：每隔 15 秒循环一次（设置页面设置）

调用预警模块，账户信息，交易模块，行情模块执行拿到最新数据。

发现 slow sell 满足条件，卖出 50U 下取整（注意粉尘处理，如果卖出后剩余价值小于 5U，则带上粉尘一起卖出）

发现 fast sell 满足条件，卖出 50U 下取整（注意粉尘处理，如果卖出后剩余价值小于 5U，则带上粉尘一起卖出）

重新调用预警模块，账户信息，交易模块，行情模块（如果没有卖出操作，就不需要重新调用，如果卖出了就需要重新调用，欣慰要更新账户）

阶段 2：买入阶段

根据行情模块（因为出现到行情模块列表中的都是优质标的），平均分散在前几个交易对中买（小于等于 3），如果只有一个就只买这一个，如果有 2 个就平均 2 个。

核心处理逻辑：

循环执行

调用预警模块，账户信息，交易模块，行情模块

阶段 1：卖出阶段

ship sell{

持有的某个交易对，如果当前费率大于 0.0100%，不卖，持有有利可图。

如果持有的所有交易对都有利可图（当前费率大于 0.0100%），都不卖

}

slow sell{

如果当前费率大于 0，但是小于 0.0100%（换算年化 12%左右）

判断当前持仓的基差是否小于买入时的平均基差，如果小于就可以卖（顺带基差套利），如果当前基差大于买入时的平均基差，先跳过，继续观望。

}

fast sell {

持有的某个交易对，如果出现资金费率为负、ADL 等信息，一股脑对冲卖出（优先腿依旧是现货，对冲腿为合约），单次下单 5U。

不再判断基差，再不走就要亏钱了。

}

阶段 2：买入阶段

重新调用账户信息，交易模块，行情模块等（如果 skip sell 了就不要再调用了，因为中间就间隔了几微秒，行情足够新）

注意，这里的“现金”指购买力，默认合约账户和现货账户四倍杠杆（设置页面设置），购买力= $\min(\text{合约账户} \times 4, \text{现货账户})$

```
skip buy{
```

现金不足、或者有现金但是行情模块为空（没有值得买的），那我宁愿留着现金。

```
}
```

```
scatter buy {
```

平均分散在行情列表中的前 n 个（比如 $n < 3$ ）个交易对中买。

因为有可能列表中只有 1 个，有 2 个，有 3 个，有多个。

所以 for 循环下单 $i=1, n; \{$

第 i 个标的，下单 50U。

如果余额小于 50U，则上取整。

如果没钱，跳过。

```
}
```

异常与容错处理：

接收者：自动交易需要日志存入数据库，同时需要发送邮件，通知用户自己执行了哪些操作。

输出数据：

模块设计考量与权衡：

粉尘价格处理：本次买入卖出时如果发现执行之后，剩余的币等值或者剩余的 usdt 小于 5U，则本次买入卖出携带剩余粉尘。

麦穗理论：不要贪心。

牛吃草，吃几口抬头看一下（下几次原子单，抬头看一下行情有无变动，毕竟是自动交易，7x24 小时都在跑，不争一分一秒，谨慎一

些，没有错。所以我才以 50U 为一组，以 5U 为原子单位（当然所有的参数都可以调整）

7、设置模块

在前端页面设置值，存入数据库。其他所有模块从数据库中读取，同时其他读取参数的模块，要把参数显示出来。不要隐藏起来。